



UNIVERSIDAD AUTÓNOMA DEL ESTADO DE HIDALGO

Instituto de Ciencias Básicas e Ingeniería

Centro de Investigación en Tecnologías de Información y Sistemas

Licenciatura en Ciencias Computacionales

Criptografía (Optativa II) Octavo semestre

Reporte de Tarea 09

“Uso Servidor HTTP embebido con Wifi”

Trabajo realizado por:
Lara Hernandez Carlos Alberto
Guerrero Olvera Angelica

Resumen: *En este trabajo se replicó e implementó la configuración de un servidor web embebido utilizando una placa NodeMCU v3 (ESP8266) programada con PlatformIO en VS Code. El sistema permite al usuario conectarse desde un teléfono Android en la red local (mediante la IP 192.168.1.166) a un dashboard web, enviando comandos asíncronos para dibujar geometrías personalizadas (logos "Gears" y "Halo") en una pantalla OLED de 0.96 pulgadas con una resolución de 128x64 píxeles. Adicionalmente, se incluye el análisis de una arquitectura de Internet de las Cosas (IoT) orientada a la nube mediante el uso de Firebase Realtime Database y GitHub Pages, discutiendo su viabilidad para el Arduino UNO R4 WiFi*

1. Introducción

1.1 Planteamiento del problema

El control y monitoreo de dispositivos de hardware a través de un navegador web requiere la configuración de microcontroladores capaces de procesar protocolos HTTP. El desafío consiste en construir una arquitectura sólida para redes de área local (LAN) y contrastar sus limitaciones de acceso exterior frente a las nuevas arquitecturas de Internet de las Cosas (IoT) que permiten el acceso global.

1.2 Contexto del problema

En el curso se han evaluado tarjetas con conectividad Wi-Fi, como la serie ESP8266 y el Arduino UNO R4 WiFi. Al desplegar un servidor web básico de forma embebida, el acceso se restringe únicamente a los usuarios dentro de la misma red local. Para lograr un control ubicuo, es necesario integrar bases de datos intermediarias y plataformas en la nube.

1.3 Objetivos

- Replicar la lógica de interacción de un control vía web adaptando un servidor embebido en una NodeMCU v3 ESP8266.
- Lograr la comunicación entre un smartphone Android y la NodeMCU empleando la red WiFi local (IP: 192.168.1.166).
- Visualizar resultados en un display OLED de 0.96" (128x64 píxeles).
- Analizar un modelo de arquitectura IoT en la nube para el Arduino UNO R4 WiFi.

1.4 Propuesta de solución

Se propone el uso de PlatformIO para administrar las dependencias (Adafruit GFX, Adafruit SSD1306, ESP8266WebServer). La solución se divide en un frontend dinámico (WebPage.h) y un backend de control (main.cpp) que gestiona los endpoints HTTP.

2. Metodología y teoría de solución

La práctica se basa en la arquitectura Cliente-Servidor LAN:

1. Servidor Embebido: La NodeMCU v3 actúa como servidor escuchando solicitudes HTTP en el puerto 80.
2. Cliente Web Asíncrono: La interfaz web utiliza la API fetch() para enviar órdenes sin recargar la página, mejorando la experiencia de usuario.
3. Visualización OLED: En lugar de cargar mapas de bits pesados, se utilizan primitivas gráficas (drawCircle, drawLine) para renderizar los logos de forma eficiente, aprovechando la resolución de 128x64 píxeles de la pantalla de 0.9".
4. Respecto al Análisis IoT Global: Se estudia la transición de un servidor local a uno basado en la nube. En este esquema, el microcontrolador (Arduino UNO R4 WiFi) actúa como cliente de una base de datos en tiempo real (Firebase), permitiendo el control desde cualquier parte del mundo sin necesidad de redireccionamiento de puertos (Port Forwarding).

3. Solución propuesta

3.1 Materiales y conexiones

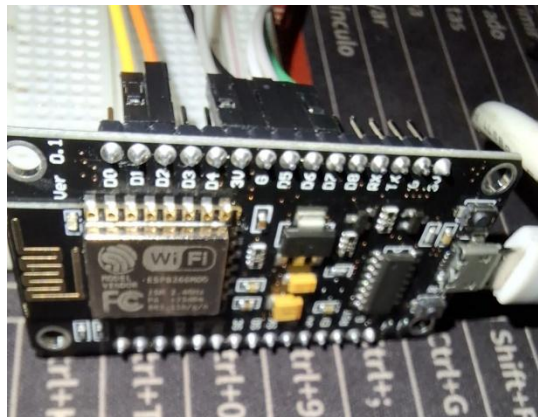
Componente	Detalle Técnico
Microcontrolador	NodeMCU v3 (ESP8266)
Pantalla	OLED 0.96" (128x64 píxeles, controlador SSD1306)
Conectividad	Conexión exitosa a través de IP local 192.168.1.166
Endpoints	Se crearon las rutas /gears y /halo para activar las funciones de dibujo correspondientes.
Entorno	VS Code + PlatformIO

3.2 Algoritmo de operación

1. Splash Screen: Se despliega el mensaje "Tar-09 WIFI HTTP SERVER" durante 3000ms para confirmar el estado del buffer de video.



2. Handshake WiFi: La NodeMCU intenta conectar al SSID definido en secrets.h. Mientras conecta, se muestra una animación de puntos en la OLED.



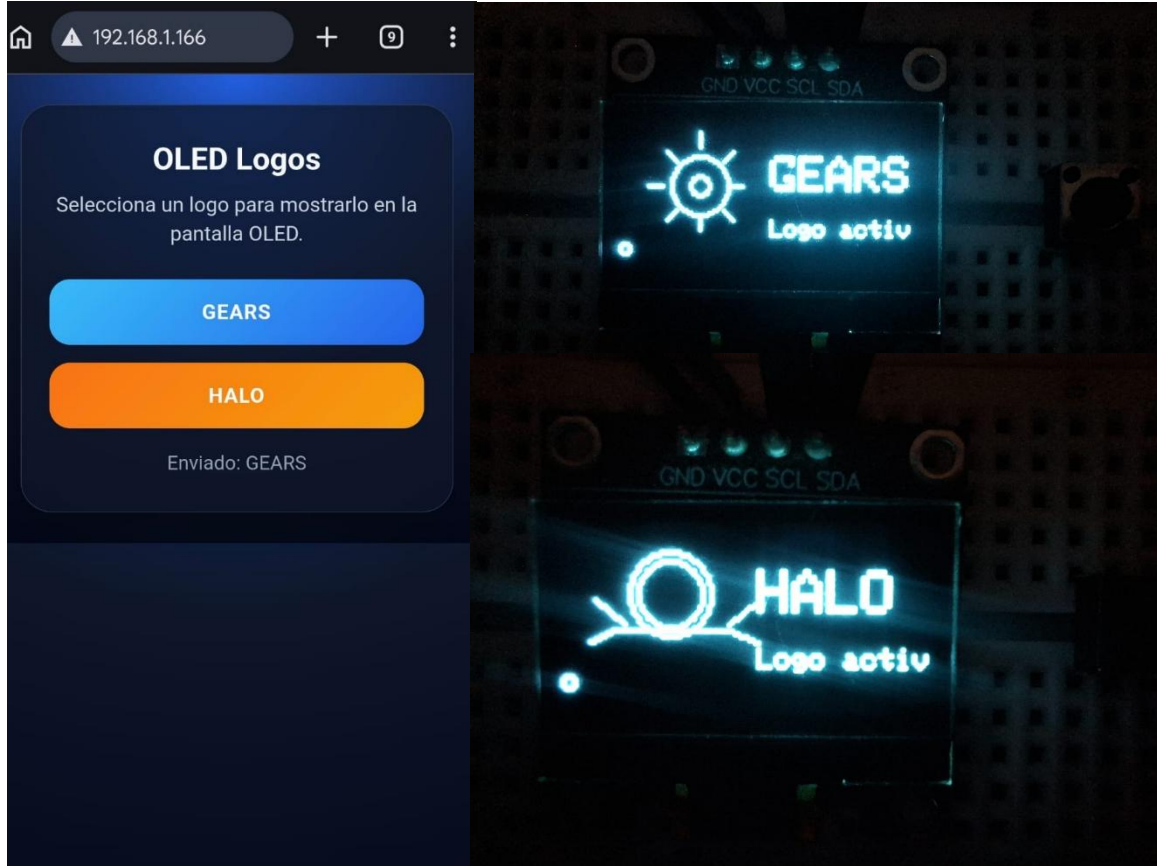
3. Despliegue de IP: Una vez obtenida la conexión, el sistema imprime la IP local (192.168.1.166) en pantalla para que el usuario sepa dónde navegar..



4. Recepción Configuración de Endpoints:

Se mapean las funciones de callback:

- a. GET /: Envía el contenido del buffer HTML_CONTENT.
- b. GET /gears: Ejecuta las primitivas de dibujo para el engrane y refresca el display.
- c. GET /halo: Ejecuta el dibujo del halo y refresca el display. el teléfono envía un mensaje, este se lee y se despliega en la pantalla OLED por un periodo de 2 segundos.



3.3 Discusión sobre Arduino UNO R4 WiFi

Para implementar el control global en el Arduino UNO R4:

- Integración de Firebase: La placa sincronizaría variables de estado directamente con la nube.
- Dashboard Externo: El HTML se alojaría en GitHub Pages para liberar memoria flash en el microcontrolador.
- Seguridad: Uso de autenticación por tokens para evitar accesos no autorizados.

3.4 Observaciones experimentales

- Se detectó que el uso de la API `fetch()` en el frontend evita el parpadeo de la web al enviar comandos, manteniendo la conexión estable.
- La resolución real ocupada es de 128x64 píxeles, optimizada para que los logos no se corten en los bordes de la pantalla de 0.9".

4. Conclusión de pantallas, ejercicios, prácticas y resultados

- Se confirmó que la pantalla OLED tiene una resolución de 128x64 píxeles, lo cual permite un nivel de detalle adecuado para los logos "Gears" y "Halo".
- La latencia de respuesta entre el teléfono Android y la placa fue mínima.
- PlatformIO demostró ser una herramienta superior al IDE de Arduino para la gestión de proyectos con múltiples archivos de cabecera.

Conclusión general

El desarrollo de servidores web locales es el primer paso esencial para entender la comunicación máquina-usuario. Sin embargo, para aplicaciones profesionales de IoT, la arquitectura debe evolucionar hacia servicios en la nube como Firebase, lo que permite una escalabilidad global y una seguridad robusta, tal como se analizó para el caso del Arduino UNO R4 WiFi.

Bibliografía

- Espressif Systems. ESP-FAQ: Analog-to-Digital Converter (ADC) for ESP8266.
- ESP8266 Arduino Core Documentation. Reference: Analog input and analogRead(A0).
- López Morales, Virgilio. Criptografía (Optativa II), notas del curso, Enero-Junio 2024.
- Paar, C., & Pelzl, J. (2010). Understanding Cryptography. Springer.
- Tech Talkies. (2024). Uno R4 webserver | Detailed guide and code. [Video]. Recuperado de: <https://www.youtube.com/watch?v=tiB8xECOK7I>
- Tech StudyCell. (2025). ESP32 IoT Home Automation | Firebase Realtime. [Video]. Recuperado de: <https://www.youtube.com/watch?v=xIdX0eosdP4>

Apéndice. Código fuente utilizado

El siguiente código corresponde a la implementación compartida para la ESP8266. Se conserva como evidencia técnica y como base para futuras mejoras.

Links de códigos: <https://github.com/KaarLarax/Cryptography/tree/main/practices/tar-09>

Link General de la materia: <https://github.com/KaarLarax/Cryptography/>